LAB4

Finite State Machines with time considerations

OBJECTIVE

The objective of this laboratory is to:

1. Design a more complex state machines with time considerations

a Recommendation: read Ashenden 4.3.1

Traffic light controller

Create a state diagram and Verilog code for the traffic lights at the intersection of a highway and a cross-street. Assume that the lights on both sides of the street are tied together, so there are 2 lights (one for the highway and one for the cross-street). Each light has 3 outputs (red, yellow, green), so the system has a total of 6 outputs (HR, HY, HG, CR,CY, CG – H for Highway and C for Cross street). The system has 2 inputs - the first is a sensor (S) on the cross-street that indicates somebody is waiting at the cross-street. The second is a continuously running 3-second timer (TC). Assume the timer is starting counting down from a pre-defined value (localparam) whenever you reach a new state (see more about the counter below). The system should normally have the highway **green** lights on (HG) and the cross-street **red** lights on (CR). When the sensor indicates that there is a person waiting at the cross-street, the system should wait for the 3-second timer, then transition to the highway yellow light (HY), wait three more seconds, transition to the highway red light (HR). After three more seconds, the cross-street green light should come on (CG). The system should stay in this state until the sensor indicates that there is nobody left waiting at the cross-street. It should then wait for the timer, and begin the process of transitioning back to the highway green (HG). Red should stay in this state for 20 seconds before it changes again.

Hint: you have 6 states, Highway (G,Y,R) and Cross street (G,Y,R). you can call them HG, HY,HR and CG,CY and CR. Inputs are: s (for sensor), tc (for timer completion), clk and reset. Outputs are: hr,hy,hg,cr,cy,cg. Assume that reset signal (async) bring you to the state HG. Use paramenter (localparam) for the states names (e.g., HG = 0, HY = 1, etc.) .

Implement the timer in your circuit rather than as an input to the system. Assume: You have a 50Mhz clock. Timer counts down (so it has a reset value – a "pre-set", by a "timer_reset" signal) and enable "timer_enable" for count down when needed). Use localparam for the timer reset ("pre-set") value.

Add in a 20 second minimum – so that once a light has turned green, it will stay green for at least 20 seconds. The sensor activation will still require three seconds to activate, but there must be at least 20 seconds before the light begins changing.

Hint: use another 20 second timer.. (timer20 reg). We can't use the previous 3 sec timer as the time is independent. Use timer20_enable and timer20_reset, and tc_20 (when timer reaches 0) signals. We don't want this counter to reset (not to get again the 20 second value for count down) when it gets to the end because we want to know that it has reached the end, not keep cycling. We will use the reset to restart the timer. The 3 second timer should work the same way …

Deliverables

1. State machine diagrams for the implementation, showing transitions based on input conditions.
2. A table of all the states vs. inputs yielding the transitions and related outputs.
3. A Verilog code for the traffic light controller and a testbench to simulate it.
4. Show in simulation: state after reset, what happens if S asserted any time in HG and CG states, show transition according to timers

1. **Submit:**

   - State transitions diagrams for the traffic light controllers.
   - Verilog files: traffic_lights.v, Traffic_lights_tb.v
   - Printout of the results
   - Waveforms images.